

Ejercicio 1

Inciso 1

- $T1 = \frac{8}{20} = 0.4$
- $T2 = \frac{10}{50} = 0.2$
- $T3 = \frac{5}{25} = 0.2$
- $T4 = \frac{15}{100} = 0.15$

Inciso 2

El factor de utilización total sería la suma de los 4 factores anteriores: $0.4 + 0.2 + 0.2 + 0.15 = 0.95$

Inciso 3

Se encuentra en una zona insegura del scheduling (*dangerous*), porque se encuentra entre el 83% y 99%.

Inciso 4

Trataría de optimizar la tarea **T1**, ya que su tiempo de ejecución es muy grande a su periodo, con hacer su periodo **50ms** como en el tarea **T2**, el sistema total tendría un factor de utilización: 0.71, que estaría en la zona **cuestionable**, que se mucho mejor que antes.

Ejercicio 2

Inciso 1

No, no debe configurarse el TICK del sistema a 100us, ya que esto produciría que el scheduler se dispare cada 100us, haciendo más alta la carga del sistema operativo, y como el programa ya está demandando mucha carga, esto haría que el microcontrolador se exceda en capacidad de procesamiento. En general **no** es buena idea cambiar el TICK del sistema para cumplir una demanda de muestreo.

Inciso 2

Aumentar excesivamente la frecuencia del TICK hace que nunca se ejecuten las tareas, ya que se terminan cambiando continuamente sin que se puedan lograr ejecutar por completo, excediendo en capacidad de procesamiento al microcontrolador.

Inciso 3

Se podría configurar un timer que genere una interrupción y procesar la muestra “*por fuera*” de freeRTOS (ejemplo de la gaviota en la teoría).

Ejercicio 3

Inciso 1

Cuando ambas tareas tratan de acceder al mismo recurso pueden ocurrir un problema llamado *racing condition*, efectivamente haciendo que no se transmitan los mensajes correctamente (podría intercambiarse letras o “*pisarse*” las palabras).

Inciso 2

Se podría hacer uso de las **Queues** de FreeRTOS, haciendo una cola, donde las tareas actúan como productores de la información (mensajes), y el UART actúa como un consumidor de la misma. Otra

forma sería con **mutex** o **semáforos**, bloqueando el recurso cuando uno esté accediendo, así el otro no trata de acceder al mismo tiempo, dependiendo del resto del programa y la aplicación puede ser conveniente la opción de las **Queues** o esta última.

Inciso 3

La inversión de prioridades, es cuando una tarea tiene un recurso bloqueado (con un mutex por ejemplo), y otra tarea de mayor prioridad quiere ejecutarse y acceder a ese recurso, pero como la primera tarea estaba haciendo uso del recurso, la tarea de mayor prioridad “se queda esperando” a que la de menor prioridad termine, que es indeseado, ya que estarían invirtiéndose las prioridades. FreeRTOS soluciona esto con lo que se denomina herencia de prioridades, efectivamente lo que se hace es que la tarea que posee el mutex, tenga la prioridad más alta de las tareas que se encuentran bloqueadas tratando de acceder al recurso, esto soluciona el problema de bloquea no acotado.

Parte práctica

Inciso 1

Consigna a

Se implementarán 3 tareas, las llamaré:

- **MuestreoTask**: se encargará de realizar el muestreo del ADC cada 5ms.
- **ProcesamientoTask**: se encargará de cargar los datos en un arreglo y luego generar el promedio.
- **UITask**: se encargará de tomar los eventos y datos y enviarlos por UART al usuario cada 5 segundos.

Consigna b

Implementaré dos **Queues**. Una (**Q1**) que se encargue de almacenar las muestras creadas por **MuestreoTask**, y otra (**Q2**) que se empleará para realizar la mensajería con **UITask**.

- **Q1** tendrá un tamaño de 400 muestras, equivalente a $(400 \text{ muestras}) * \frac{5 \text{ ms}}{\text{muestras}} = 2\text{s}$.
- **Q2** tendrá un tamaño de 10 elementos, es más que suficiente ya que se producen notificaciones cada 2 segundos y se imprimen cada 5 segundos.

El elemento de **Q2** será un struct de 4 Bytes, donde 2B serán para un enum indicando el tipo de mensaje, y los otros 2B serán para un payload, es decir un dato.

Consigna b

No emplearía ningún otro elemento de sincronización, ya que **Q2** se encargaría de evitar problemas con la UART.

Consigna c

Crearía una variable global que almacene el estado de la alarma.

Consigna e

- **MuestreoTask**: 512B, ya que es el mínimo y esta tarea sólo empleará **Queues**, esas funciones de RTOS ocupan alrededor de 160B, y luego para muestrear no se requiere mucha más memoria.
- **ProcesamientoTask**: 1024B, ya que crearé un buffer local que almacene $\frac{2 \text{ segundos}}{5 \text{ ms por muestra}} = 400$ muestras de 2B cada una (ya que el ADC es de 12bits), es decir un buffer de 800B, además al igual que antes al usar funciones del freeRTOS necesito un poco más de memoria, así que en principio asignaría 1024B, pero luego verificaría si es suficiente con “Build Analyzer” y “Static Stack Analyzer”.
- **UITask**: 512B, sólo necesita acceder al UART y funciones del FreeRTOS, quizá requieran más, pero en principio sólo es eso.

- **Heap:** Considerando que se requieren las dos **Queues**, pero se asignan de forma estática (por fuera del Heap), y que se contienen las 4 tareas, todo suma a 2650B aproximadamente, dejando un margen.

Consigna f

- **MuestreoTask:** $\frac{0.5}{5} = 0.1$ (estoy considerando que puede llegar a tardar 500us en un peor caso)
- **ProcesamientoTask:** $\frac{200ms}{2s} = 0.1$ (considerando que procesar 400 muestras lleve 200ms)
- **UITask:** $\frac{200ms}{5s} = 0.04$ (considerando que transmitir por UART lleve 200ms)

Creo que fui generoso con los tiempos de ejecución (es decir consideré más tiempo del que realmente tardarían), y el factor de utilización total se estima a $FU = 0.1 + 0.1 + 0.04 = 0.24$, que dado lo que hace el sistema, parece acorde.

Consigna g

- **MuestreoTask:** Prioridad máxima (High), ya que quiero que el tiempo de muestreo se cumpla siempre
- **ProcesamientoTask:** Prioridad media (Medium), ya que es más crítico que se calculen los promedios a que se transmita el UART
- **UITask:** Prioridad baja (Low), ya que la prioridad del UART no la considero más importante que **ProcesamientoTask**

Inciso 2

Todas las capturas están en una carpeta, no tuve tiempo de hacerlo más prolijo.